



دانشگاه صنعتی امیرکبیر
(پلی تکنیک تهران)

برنامه سازی پیشرفته و کارگاه

کار با فایل

نگارش:

پیام دیوسالار، آرمان حسینی، مهدی جعفری

استاد درس:

دکتر روح الله احمدیان

دانشکده ریاضی و علوم کامپیوتر

دانشگاه صنعتی امیرکبیر

بهار ۱۴۰۵

فهرست مطالب

۲	۱	کار با فایل
۲	۱.۱	مقدمه
۲	۲.۱	کلاس فایل (File Class) و آدرس‌دهی
۳	۱.۲.۱	آدرس‌دهی فایل‌ها در جاوا (File Paths)
۴	۲.۲.۱	متدهای کاربردی کلاس File
۵	۳.۱	جریان داده‌ها Stream
۶	۴.۱	Byte stream
۶	۱.۴.۱	Input stream و Output stream
۷	۵.۱	Character Stream
۸	۱.۵.۱	FileReader
۹	۲.۵.۱	FileWriter
۹	۳.۵.۱	معجزه Buffer و BufferedWriter
۱۰	۶.۱	خواندن از فایل با کلاس Scanner
۱۲	۱.۶.۱	کلاس Scanner با String
۱۳	۷.۱	نوشتن به فایل با کلاس PrintStream
۱۳	۱.۷.۱	ورودی و خروجی در کلاس System
۱۴	۸.۱	روش مدرن کار با فایل‌ها (NIO.2)
۱۵	۹.۱	چیزی که یاد گرفتیم
۱۵	۱۰.۱	منابع بیشتر

فصل ۱ کار با فایل

۱.۱ مقدمه

میدونید یا قراره بدونید که توی خیلی از برنامه‌ها با داده‌هایی سر و کار داریم که الزاماً ورودی همون لحظه ترمینال نیستن و نیاز دارید از داده‌هایی که قبلاً داریم استفاده کنید، یا ممکنه بخواین ورودی‌هاتون رو از هر رابط کاربری‌ای که می‌گیرید در یک جای جداگانه ذخیره کنید. حتی ممکنه لازم باشه چیزی خارج از کدهاتون توی برنامه اصلی استفاده بشه، مثلاً اگه یادتون باشه برای پروژه منوی کافه باید یه سری عکس از خارج برنامه لود می‌کردین و با دادن آدرسش به کدتون اون‌ها رو استفاده می‌کردین.

به همه دلایل بالا و خیلی از دلایل دیگه ما نیاز داریم که دیتاهمون رو با یه روشی ذخیره کنیم و بعداً بازخوانیشون کنیم تا به یه دردی بخورن. برای این کار ما نیاز داریم بتونیم با این فایل‌ها کار کنیم و از اون‌ها چیزایی رو بخونیم یا توی اون فایل‌ها تغییراتی ایجاد کنیم و اطلاعات رو توشون بنویسیم. همون‌طور که می‌دونید مفاهیم خوندن و نوشتن چه در جاوا چه در هر زبان یا برنامه‌ای که با کامپیوتر سر و کار داره به مفهوم ورودی و خروجی یا به طور مختصر I/O گره خورده. توی جاوا یعنی هرجایی که بخوای اطلاعاتی وارد برنامه‌ات کنی (مثل خوندن از یه فایل)، یا از برنامه‌ات بریزی بیرون (مثل نوشتن توی فایل). برای این کار ما از یه پکیج توی جاوا استفاده می‌کنیم به اسم `java.io`.

۲.۱ کلاس فایل (File Class) و آدرس‌دهی

قبل از اینکه بریم سراغ خوندن و نوشتن از فایل‌ها، نیاز داریم که بتونیم بهشون توی سیستم‌عامل اشاره کنیم. کلاس `File` توی پکیج `java.io` قرار داره و دقیقاً برای کار با فایل‌ها و پوشه‌ها (دایرکتوری‌ها) استفاده می‌شه.

حالا آیا با کلاس `File` می‌تونیم بخونیم یا بنویسیم توی فایل؟ نه! این کلاس خودش مستقیماً برای خوندن و نوشتن استفاده نمی‌شه. فقط برای نمایش، ایجاد، حذف، تغییر نام و بررسی ویژگی‌های فایل و پوشه استفاده می‌شه. ساختن یک شی از کلاس `File`:

```
File myFile = new File("path/to/file.txt");
```

پس نیاز داریم آدرس بدیم (که احتمالاً خیلیاتون از چالش‌هاش خبر دارید).

۱.۲.۱ آدرس‌دهی فایل‌ها در جاوا (File Paths)

وقتی می‌خواهیم به فایل روی توی جاوا باز کنیم، باید به مسیر (Path) بهش بدیم. این مسیر می‌تونه مطلق (Absolute Path) یا نسبی (Relative Path) باشه. بیایم دونه دونه بررسی کنیم.

آدرس مطلق (Absolute Path)

آدرس کامل از ریشه‌ی سیستم‌عامل تا خود فایل یا پوشه. ویندوز:

```
File file = new File("C:\\Users\\Mahdi\\Documents\\file.txt");
```

لینوکس/مک:

```
File file = new File("/home/mahdi/documents/file.txt");
```

نکته: توی ویندوز چون \ کاراکتر خاص (escape character) هست، باید دوتا بذاریم \\. همچنین آدرس مطلق همیشه یکیه، فرقی نمی‌کنه پروژه‌ت کجا اجرا شه. ولی ممکنه روی سیستم‌های مختلف جواب نده. (مثلاً آدرس ویندوز روی مک نمی‌خونه یا روی سیستم ویندوزی که آدرس‌هاش فرق داره جواب نمیده.)

آدرس نسبی (Relative Path)

نسبت به مکانی که برنامه‌ت از اون اجرا می‌شه سنجیده می‌شه. این‌جوری قابل حمل‌تره. مثلاً فرض کن به فایل به اسم data.txt توی پوشه‌ی پروژه‌ت داری. اون وقت این‌جوری می‌نویسی: (توجه کنین که منظور از پوشه پروژه root دایرکتوریه که بسته به IDE سیستم یا نوع باز کردن پروژه‌تون می‌تونه متفاوت باشه)

```
File file = new File("data.txt");
```

یا اگه فایل توی به زیرپوشه‌ی خاص باشه:

```
File file = new File("resources/data.txt");
```

به طور دقیق‌تر این مسیر نسبت به دایرکتوری اجرایی (working directory) سنجیده می‌شه، معمولاً جایی که فایل class یا jar اجرا می‌شه.

ترکیب مسیرها با File.separator

اگه بخوای مسیر رو طوری بسازی که روی همه‌ی سیستم‌عامل‌ها کارکنه، بهتره از File.separator استفاده کنی: توی ویندوز File.separator می‌شه: \ توی لینوکس و مک میشه: /

همچنین برای گرفتن مسیر فایل ها می توانیم از روش های زیر استفاده کنیم: گرفتن مسیر کامل فایل:

```
System.out.println(file.getAbsolutePath());
```

گرفتن مسیر والد (پوشه ی بالا):

```
System.out.println(file.getParent());
```

گرفتن مسیر اجرایی فعلی:

```
System.out.println(System.getProperty("user.dir"));
```

جمع بندی آدرس دهی

معایب	مزیت	آدرس نوع
سیستم ها بین نیست قابل حمل	قطعی و دقیق	مطلق
باشه وابسته برنامه اجرای دایرکتوری به ممکنه	ساده قابل حمل،	نسبی
نوشتن در پیچیده تر کمی	امن تر حمل، قابل	با File.separator

خب بیایم برگردیم به کلاس فایل مون. آبجکت کلاسمون رو ساختیم، بیایم و ببینیم این آبجکت چه متدهایی داره که به دردمون می خوره:

۲.۲.۱ متدهای کاربردی کلاس File

- متد `createNewFile()`: ساخت فایل جدید. این متد ممکنه `IOException` بندازه، پس باید توی `try-catch` بذارش.
- متدهای `mkdir()` و `mkdirs()`: ساخت پوشه (دومی فولدرهای تو در تو می سازه).
- متد `exists()`: بررسی وجود فایل یا پوشه.
- متد `delete()`: حذف فایل یا پوشه.
- متدهای `getName()` و `getPath()` و `getAbsolutePath()`: گرفتن نام و مسیر فایل.
- متد `length()`: گرفتن سایز فایل به بایت.
- متدهای `list()` و `listFiles()`: لیست کردن محتویات پوشه.
- متد `renameTo(File newName)`: تغییر نام یا انتقال.

پس با همه چیزایی که یاد گرفتیم به مثال کامل برای ساخت فایل، بررسی وجود و حذف رو می‌بینیم:

```
import java.io.File;
import java.io.IOException;

public class FileExample {
    public static void main(String[] args) {
        try {
            File myFile = new File("example.txt");
            if (myFile.createNewFile()) {
                System.out.println("      : " + myFile.getName());
            } else {
                System.out.println("      " );(

            }

            System.out.println("      " + myFile.exists());
            System.out.println(" : " + myFile.getAbsolutePath());

            if (myFile.delete()) {
                System.out.println("      " );(
            } else {
                System.out.println("      " );(
            }
        } catch (IOException e) {
            System.out.println("      " );(
            e.printStackTrace();
        }
    }
}
```

۳.۱ جریان داده‌ها Stream

تا حالا به این فکر کردین که وقتی به فایل رو با کلاس File پیدا کردیم، برنامه چطوری اون اطلاعاتو می‌خونه؟ این جاست که بحث Stream یا همون جریان داده‌ها میاد وسط. در واقع به استریم رو می‌شه مثل یه رودخونه یا آبراهه از داده‌ها در نظر گرفت که اطلاعات مثل آب توی اون جاری‌ان و از یه منبع شروع به حرکت می‌کنن و وارد برنامه می‌شن یا برعکس از برنامه ریخته می‌شن به یه مقصدی. اگه بخوایم به تعریف شسته‌رفته‌تر و رسمی‌تر از stream داشته باشیم می‌شه گفت که stream نشون‌دهنده‌ی جریانی از داده‌هاست که یه طرف اون‌ها یه نویسنده یا writer و توی سر دیگه‌ش یه

خواننده یا reader داریم.

استریم‌ها تو جاوا دو نوع اصلی دارن: **Byte stream** و **Character stream**.

۴.۱ Byte stream

برای وقتی هست که داده‌ها متنی نیستن، یعنی قراره با فایل‌های باینری مثل عکس، ویدیو، فایل‌های PDF، موسیقی، یا هر چیزی که ساختار عددی یا خاص داره کار کنی. که شامل کلاس‌های اصلی زیر می‌باشد:

۱.۴.۱ Output stream و Input stream

کلاس‌های ابسترکتی برای خوندن و نوشتن داده‌های باینری هستن که تمام کلاس‌های کار با داده‌های باینری ازشون مشتق می‌شن. وظیفه‌شون اینه که یه سری متد پایه مثل `read` یا `write` برای کار با بایت‌ها بدن و بقیه‌ی کلاس‌ها بیان و اونا رو پیاده‌سازی کنن. متدهای مهمشون شامل `read()`، `write()`، `flush()` و `close()` هست.

وقتی می‌خوایم داده‌های باینری رو بخونیم یا بنویسیم این دوتا کلاس ابسترکت به تنهایی کافی نیستن و در عمل به زیرکلاس‌های تخصصی‌تری نیاز داریم. از این زیرکلاس‌ها به `FileInputStream` و `FileOutputStream` اشاره می‌کنیم.

FileInputStream

این کلاس برای وقتی هست که بخوای داده‌های خام از یه فایل بخونی. مثلاً بخوای یه تصویر، یه فایل صوتی یا هر فایل غیرمتنی رو بخونی.

نکته بسیار مهم: مدیریت منابع با Try-With-Resources

در گذشته برای کار با فایل‌ها، برنامه‌نویس‌ها مجبور بودن در انتهای کد متد `close()` رو دستی صدا بزنن. اما این کار یک ایراد بزرگ داشت: اگر قبل از رسیدن به خط `close` اروری رخ می‌داد، برنامه وارد `catch` می‌شد و فایل برای همیشه در حافظه سیستم عامل باز می‌موند! (Resource Leak). در جاوای مدرن (از جاوا ۷ به بعد) ما از ساختار `Try-With-Resources` استفاده می‌کنیم. هر فایلی که داخل پرانتز جلوی `try` باز بشه، در انتهای بلوک به طور کاملاً خودکار بسته می‌شه، حتی اگه ارور بده! ما در تمام مثال‌های این داکيومنت از این استاندارد استفاده می‌کنیم. فرض کن می‌خوای یه فایل تصویری رو بخونی و اندازه‌اش رو در خروجی نشون بدی (با شمارش تعداد بایت‌ها):

```
import java.io.FileInputStream;
import java.io.IOException;

public class Main {
    public static void main(String[] args) {
        // try (
        try (FileInputStream fis = new FileInputStream("image.jpg")) {
```

```

        int count = 0;
        while (fis.read() != -1) {
            count++;
        }
        System.out.println("File size in bytes: " + count);
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

متدهای مهم FileInputStream:

- `int read()`: به بایت از فایل می‌خواند و به صورت عدد بین 0 تا 255 برمی‌گرداند. اگر به انتهای فایل برسد، -1 برمی‌گرداند. مثلاً اگر خروجی این متد 65 باشد یعنی کد ما A رو خوانده که کد اسکی اون 65 ه.
- `int read(byte[] b)`: این یکی به جای به بایت، به آرایه بایت رو پر می‌کنه. تعداد بایت‌های خوانده شده رو برمی‌گرداند. (سرعتش خیلی بیشتره).
- `int read(byte[] b, int off, int len)`: این متد به بخش مشخص از آرایه رو با داده پر می‌کنه، از `off` شروع می‌کنه و `len` بایت سعی می‌کنه بخونه.
- `long skip(long n)`: این متد به جای اینکه داده رو بخونه، می‌پره جلو. مثلاً اگر فایل 100 بایت باشه و `skip(10)` بزنی، می‌پره روی بایت یازدهم.
- `int available()`: می‌گه چقدر از فایل هنوز قابل خوندن، یعنی چند بایت مونده تا آخر.

۵.۱ Character Stream

حالا اگر با متن طرفی، یعنی مثلاً فایل‌هایی مثل `.txt`، کدهای منبع، یا هر چیز قابل خوندن (به هر زبانی که باشه)، باید بری سراغ `Character Stream`. فرض کنید قصد نوشتن یه برنامه برای دریافت اطلاعات اشخاص و ثبتشون توی یه فایل متنی داریم. اول از همه نیاز داریم یه ساختاری داشته باشیم که حین اجرای برنامه یه ساختار برای پردازش اطلاعات ورودی داشته باشیم که بعدش اون رو توی فایل خروجی ذخیره کنیم. پس کلاس زیر رو با رعایت کیسوله‌سازی تعریف می‌کنیم:

```

public class Student {
    private String firstName;
    private String familyName;
}

```

```

private int age;
private int entryYear;
private double gpa;

public Student(String firstName, String familyName, int age, int
    ↪ entryYear, double gpa) {
    this.firstName = firstName;
    this.familyName = familyName;
    this.age = age;
    this.entryYear = entryYear;
    this.gpa = gpa;
}

public String getFirstName() { return firstName; }
public String getFamilyName() { return familyName; }
public int getAge() { return age; }
public int getEntryYear() { return entryYear; }
public double getGpa() { return gpa; }
}

```

(نکته: در پروژه‌های واقعی اگر قصد دارید کل آبجکت Student رو به همین شکل در یک فایل ذخیره کنید تا بعداً دقیقاً به شکل یک آبجکت بازبیش کنید، نیازی نیست اون رو به متن تبدیل کنید! می‌تونید از مبحثی به نام Serialization و کلاس ObjectOutputStream استفاده کنید که یکی از مباحث جذاب جاوای پیشرفته است.)

اما اینجا می‌خوایم اطلاعات رو به صورت یک فایل متنی خوانا ثبت کنیم. کلاس‌های پایه‌ی Char-acter Streams شامل Reader و Writer هستن. زیرکلاس‌های مهم اون‌ها شامل FileReader و FileWriter می‌شن.

1.5.1 FileReader

برای خواندن فایل‌های متنی استفاده می‌شه. پشت‌صحنه از رمزگذاری پیش‌فرض سیستم برای تبدیل بایت‌ها به کاراکتر استفاده می‌کنه.

```

//      try-with-resources
try (FileReader fr = new FileReader("text.txt")) {
    int c;
    while ((c = fr.read()) != -1) {
        System.out.print((char) c);
    }
} catch (IOException e) {
    e.printStackTrace();
}

```

}

۲.۵.۱ FileWriter

برای نوشتن متن به فایل استفاده می‌شود. اگر فایل وجود نداشته باشد، خودش می‌سازد؛ اگر هم باشد، پیش‌فرض پاک می‌کند و از نو می‌نویسد. همچنین کانستراکتورش به پارامتر `true` (برای `append` mode) هم می‌گیرد که باعث می‌شود داده‌های جدید به آخر فایل اضافه بشوند.

نکته مهم برای فارسی‌نویسی (انکودینگ): برای کار با فایل‌های فارسی یا کاراکترهای خاص (مثلاً ایموجی)، اگر از تنظیمات پیش‌فرض استفاده کنید ممکنه متن‌ها به هم بریزد (علامت سوال بشود). در جاوا ۱۱ به بعد (جاوا ۱۱ به بعد) نیازی به کدهای پیچیده برای انکودینگ نیست؛ کافیه `StandardCharsets.UTF_8` رو مستقیماً به سازنده پاس بدید! حالا کدمون رو برای ذخیره اطلاعات دانشجو می‌نویسیم:

```
import java.io.*;
import java.nio.charset.StandardCharsets;

public class Main {
    public static void main(String[] args) {
        Student student = new Student("Ali", "Rezaei", 20, 1400, 3.7);

        try (FileWriter writer = new FileWriter("student.txt",
            ↪ StandardCharsets.UTF_8)) {

            writer.write(student.getFirstName() + ',' +
                student.getFamilyName() + ',' +
                student.getAge() + ',' +
                student.getEntryYear() + ',' +
                student.getGpa());

        } catch (IOException ex) {
            ex.printStackTrace();
        }
    }
}
```

۳.۵.۱ معجزه Buffer و BufferedWriter

حالا بعضی اوقات لازم داریم که کلی چیز توی فایلمون بنویسیم مثلاً توی یه لوپ باشیم و می‌خوایم هربار یچیزی به ته فایل اضافه کنیم. در این حالت اگر از `FileWriter` خالی استفاده کنیم، با هر بار

write() به درخواست نوشتن مستقیم به هارد دیسک می‌فرسته که از نظر زمان و منابع خیلی خیلی گرونه و سیستم رو کند می‌کنه.

قبل از رفتن سراغ راه‌حل بیاید خیلی کوتاه ببینیم بافر چیه؟ واژه‌ی «Buffer» توی برنامه‌نویسی یعنی یه فضای موقتی توی حافظه (RAM) که داده‌ها قبل از اینکه خونده یا نوشته بشن، می‌رن اونجا. فرض کن داری یه بطری رو از شیر آب پر می‌کنی، ولی سطلی که قراره بریزی توش یه کم اون طرف‌تره. تو نمی‌تونی مستقیماً آب رو از شیر بریزی داخل سطل. پس اول با یه پارچ آب رو می‌گیری، بعد می‌ریزی توی سطل. اون پارچ همون بافره.

اینجاست که BufferedReader و BufferedWriter وارد می‌شن. اینا در واقع لایه‌هایی باهوش‌تر و سریع‌تر هستن که روی کلاس‌های قبلی سوار می‌شن.

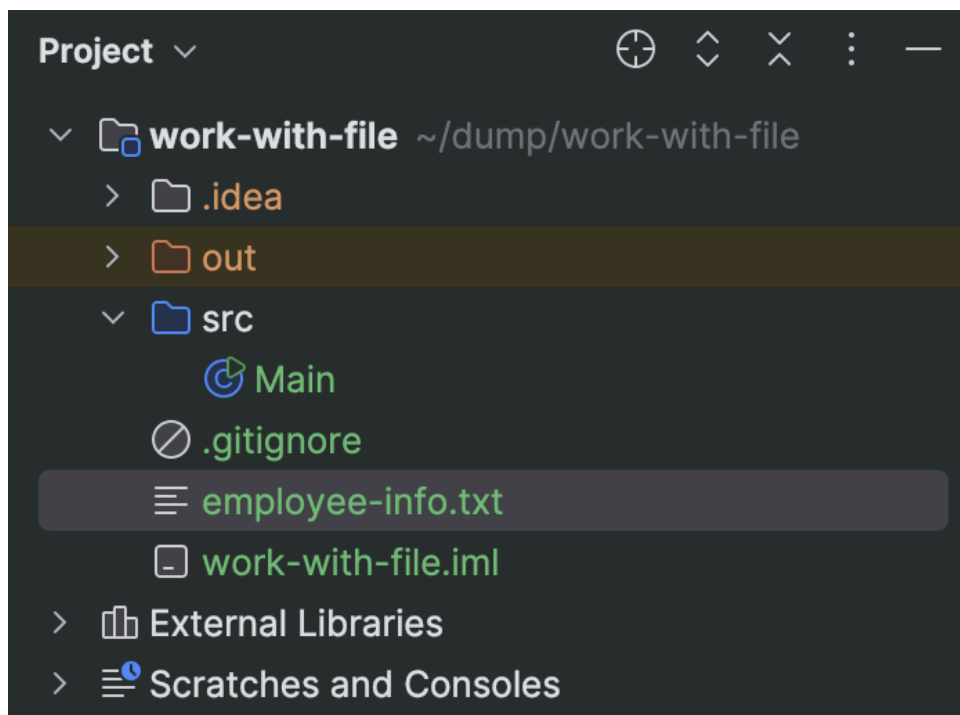
```
//          Buffer    Try-With-Resources
try (BufferedWriter bw = new BufferedWriter(new FileWriter("log.txt", true)))
{
    for (int i = 0; i < 1000; i++) {
        bw.write("Log " + i);
        bw.newLine(); //          )          BufferedWriter)
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

حالا فقط چند بار می‌ره سراغ دیسک (نه ۱۰۰۰ بار). نتیجه: سرعت بیشتر، فشار کمتر روی سیستم.

۶.۱ خواندن از فایل با کلاس Scanner

تا این‌جای کار با نحوه کار استریم‌ها آشنا شدید. حالا می‌خوایم یاد بگیریم که چطور می‌شه محتوای یک فایل رو خیلی راحت‌تر خوند و پردازش کرد.

یه پروژه جدید ایجاد کنید. توی دایرکتوری خود پروژه (خارج از دایرکتوری src)، فایل employee-info.txt رو ایجاد کنید:



توی این فایل اطلاعات زیر رو بنویسید:

```
Name: Raees - JobTitle: Riasat
Name: Gholi - JobTitle: Developer
Name: Mamad - JobTitle: Developer
```

برای خواندن این فایل، از کلاس Scanner استفاده می‌کنیم. همون Scanner ای که برای ورودی گرفتن از کاربر توی ترمینال (System.in) هم ازش استفاده می‌کردیم. این کلاس، یک کانستراکتور overload شده داره که File ورودی می‌گیره.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        File file = new File("employee-info.txt");

        try (Scanner scn = new Scanner(file)) {

            //
            while (scn.hasNextLine()) {
                String employee = scn.nextLine();
                System.out.println("Employee info: \n\t" + employee);
            }
        }
    }
}
```

```

    }

    } catch (FileNotFoundException e) {
        System.err.println("file \"" + file.getAbsolutePath() + "\" does
        ↪ not exist");
    }
}
}
}

```

اگر این کد رو اجرا کنید، خروجی زیر رو می بینید:

```

Employee info:
    Name: Raees - JobTitle: Riasat
Employee info:
    Name: Gholi - JobTitle: Developer
Employee info:
    Name: Mamad - JobTitle: Developer

```

هر کاری که قبلاً با Scanner می کردین، این جا هم می تونید بکنید. مثلاً اگر فایلی از یک سری عدد تشکیل شده باشه می تونید با استفاده از متدهای `nextInt()` و `hasNextInt()` اون ها رو بخونین.

۱.۶.۱ کلاس Scanner با String

کلاس Scanner یک کانستراکتور هم داره که ازتون String ورودی می گیره:

```
Scanner scn = new Scanner("10 20 30 40 50");
```

توی این کد Scanner به جای خوندن از فایل یا ورودی کاربر، رشته ورودی خودش رو تکه تکه (Parse) می کنه.

```

while (scn.hasNextInt()) {
    int num = scn.nextInt();
    System.out.println("Num: " + num);
}

```

۷.۱ نوشتن به فایل با کلاس `PrintStream`

برای نوشتن به فایل‌ها، از کلاس `PrintStream` هم استفاده می‌کنیم که امکاناتی شبیه به چاپ در ترمینال رو بهمون میده. برای این کار، اول فایل `output.txt` رو تو دایرکتوری پروژه‌تون ایجاد کنید و به شکل زیر عمل کنید:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.PrintStream;

public class Main {
    public static void main(String[] args) {
        File file = new File("output.txt");

        try (PrintStream output = new PrintStream(file)) {
            output.println("Hello World!");
            output.print("This is the last line");
        } catch (FileNotFoundException e) {
            System.out.println("cannot find \"" + file.getAbsolutePath() +
                               "↪ \"");
        }
    }
}
```

شباهت آبجکت `PrintStream` ای که درست کردین و `System.out` یک علتی داره، بیاید نگاهی به کد کلاس `System` بندازیم تا بفهمیم چرا.

۱.۷.۱ ورودی و خروجی در کلاس `System`

اگر توی کد سورس جاوا کلاس `System` رو باز کنید و کامنت‌های فیلدهای اون رو بخونین، می‌تونید تعریف این فیلدها رو ببینید:

```
public final class System {
    // Code Here...
    public static final InputStream in = null;
    public static final PrintStream out = null;
    public static final PrintStream err = null;
    // Code Here...
}
```

می‌بینید که `in`، صرفاً فیلدی از جنس `InputStream`، و کلاس `out` هم دقیقاً نوعی

PrintStream! هر کاری که با System.out می‌کردین، با متغیر output کد خودتون هم می‌تونید بکنید تا توی یک فایل چاپ کنید.

علاوه بر این دو فیلد، فیلدی مشابه out به اسم err داریم. از این فیلد می‌تونید برای خروجی دادن پیغام خطاهاتون (معمولاً با رنگ قرمز در ترمینال) استفاده کنید. مثلاً:

```
try {
    // Something can throw an exception here
} catch (Exception e) {
    System.err.println(e.getMessage());
}
```

خوبه که به عنوان یک برنامه‌نویس حرفه‌ای، همیشه خطاهاتون رو به جای System.out به System.err بفرستید.

۸.۱ روش مدرن کار با فایل‌ها (NIO.2)

تمام کلاس‌هایی که در پکیج java.io یاد گرفتید (مثل File، FileReader و ...) کلاس‌های پایه و قدیمی جاوا هستن. در بازار کار امروز و از جاوا ۷ به بعد، پکیج قدرتمندی به نام java.nio.file (یا همون NIO.2) معرفی شد که کار با فایل‌ها رو بی‌نهایت راحت‌تر می‌کنه.

توی این روش جدید، به جای کلاس File از کلاس مدرن Path، و برای انجام عملیات‌های پیچیده از کلاس جامع Files استفاده می‌شه. با این روش خیلی از کدهای طولانی رو می‌تونید فقط در یک خط بنویسید:

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.io.IOException;

public class ModernFileHandling {
    public static void main(String[] args) {
        try {
            // ... ( ... ) ...
            Files.writeString(Path.of("modern.txt"), " " "!" ; (

            // ... !
            List<String> lines = Files.readAllLines(Path.of("modern.txt"));
            System.out.println(lines.get(0));

        } catch (IOException e) {
```

```
e.printStackTrace();
    }
}
}
```

توصیه می‌کنیم برای پروژه‌های جدیدتون، حتماً مطالعه‌ای روی متدهای کلاس Files داشته باشید.

۹.۱ چیزی که یاد گرفتیم

توی این داک، ما مقدمات و استانداردهای کار با فایل توی جاوا رو یاد گرفتیم. فهمیدیم که:

- کلاس File چیه، چه کاربردی داره و آدرس‌دهی چطوری کار می‌کنه.
- استریم‌ها (Byte و Character) چه تفاوتی باهم دارن.
- چطور با استاندارد سعی ایمن (Try-with-resources) کد بنویسیم تا منابع سیستم هدر نره.
- بافر چیه و چرا برای حفظ سرعت و کارایی سیستم حیاتیه.
- چطور از کلاس‌های Scanner و PrintStream برای فایل‌ها استفاده کنیم.
- و با روش‌های مدرن java.nio آشنا شدیم.

۱۰.۱ منابع بیشتر

چیزی که ما توی این داک یاد گرفتیم، مبانی کلی برای کار با فایل‌ها و ورودی/خروجی (I/O) توی جاوا بود. جاوا امکانات خیلی گسترده‌تری برای مدیریت انواع ورودی و خروجی ارائه می‌ده (مثل ObjectOutputStream برای ذخیره مستقیم آبجکت‌ها و NIO برای برنامه‌نویسی شبکه‌ای پرسرعت). اگر دوست دارین بیشتر یاد بگیرین، می‌تونین به فصل دهم کتاب Learning Java، یعنی «File Input and Output» نگاهی بندازین.